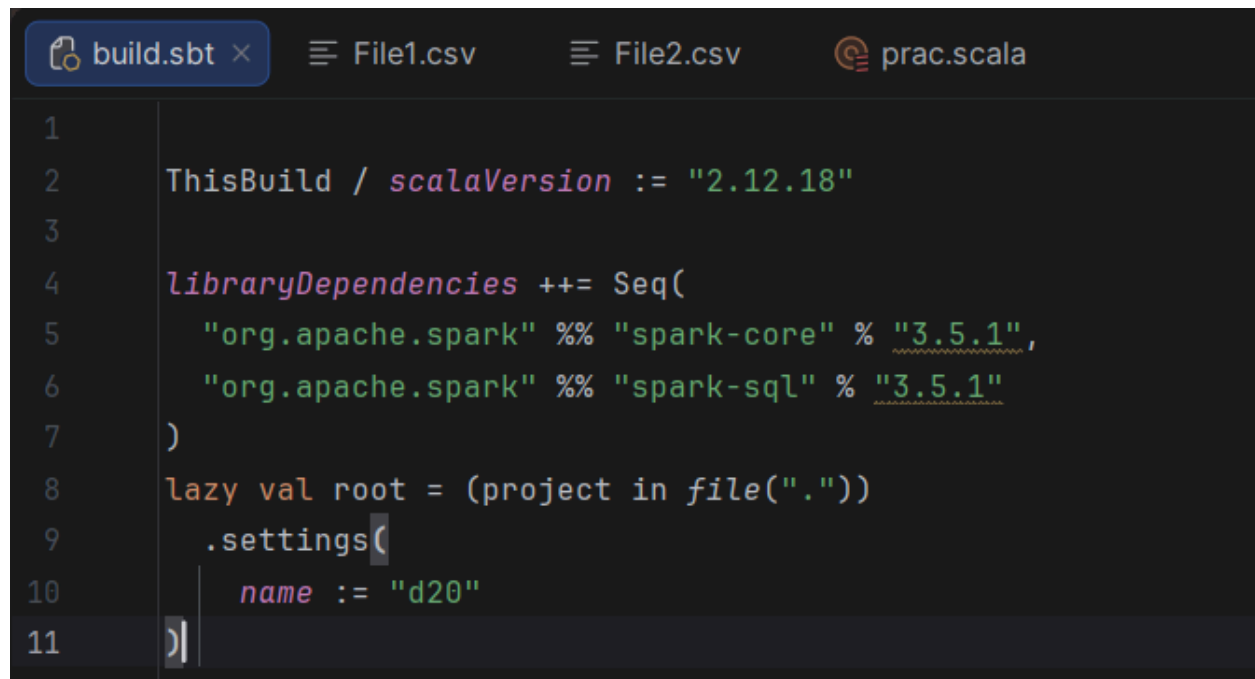


Practical - 21,22,23,24

Roll No. : S007	Name: Dhruv Gavankar
Class : SYIT	Batch : 1
Date of Assignment : 25/08/26	Date/Time of Submission : 25/08/26

Practical 21



```
1
2 ThisBuild / scalaVersion := "2.12.18"
3
4 libraryDependencies ++= Seq(
5     "org.apache.spark" %% "spark-core" % "3.5.1",
6     "org.apache.spark" %% "spark-sql" % "3.5.1"
7 )
8 lazy val root = (project in file("."))
9     .settings(
10         name := "d20"
11     )
```

```
build.sbt File1.csv File2.csv prac.scala
1 id,name
2 1,Aryan
3 2,Dhruv
4 3,Sopte
```

```
build.sbt File1.csv File2.csv prac.scala
1 id,score
2 3,234
3 2,344
4 1,444
```

Output:

```
+---+-----+-----+
| id| name|score|
+---+-----+-----+
| 1|Aryan| 444|
| 2|Dhruv| 344|
| 3|Sopte| 234|
+---+-----+-----+
```

Practical 22

Code:

```
build.sbt ×  input.txt  d27.scala
1  ThisBuild / scalaVersion := "2.12.18"
2
3  libraryDependencies += Seq(
4      "org.apache.spark" %% "spark-core" % "3.5.1",
5      "org.apache.spark" %% "spark-sql" % "3.5.1"
6  )
7  lazy val root = (project in file("."))
8      .settings(
9          name := "d24"
10     )
11
```

```
build.sbt  input.txt  d27.scala ×
1  import org.apache.spark.sql.SparkSession
2
3  ▶ object d27 extends App
4  {
5      val s = SparkSession.builder()
6          .appName("WordCountExample")
7          .master("local[*]")
8          .getOrCreate()
9      val r = s.sparkContext.textFile("input.txt")
10     val w = r.flatMap(line => line.split("\\s+"))
11     val wo = w.map(word => (word, 1))
12     val wc = wo.reduceByKey(_ + _)
13     wc.collect().foreach(println)
14     s.stop()
15 }
16
```

```
build.sbt  input.txt x d27.scala
1 Spark is easy
2 Spark is powerful
```

Output:

```
(is,2)
(easy,1)
(powerful,1)
(Spark,2)
```

Practical 23

Code:

```
build.sbt  prac.scala x superstore.csv
1 import org.apache.spark.sql.SparkSession
2
3 object prac extends App {
4   val spark = SparkSession.builder()
5     .appName("FilterCSVExample")
6     .master("local[*]")
7     .getOrCreate()
8
9   import spark.implicits._
10
11   val df = spark.read
12     .option("header", "true")
13     .option("inferSchema", "true")
14     .csv("src/main/resources/superstore.csv")
15
16   val f = df.filter($"Sales" > 10000)
17
18   println("Filtered Data:")
19   f.show()
20
21   spark.stop()
22 }
23
```

Output:

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region
2624	CA-2017-127180	10/22/2017	10/24/2017	First Class	TA-21385	Tom Ashbrook	Home Office	United States	New York City	New York	10024	East
2698	CA-2014-145317	3/18/2014	3/23/2014	Standard Class	SM-20320	Sean Miller	Home Office	United States	Jacksonville	Florida	32216	South
4191	CA-2017-166709	11/17/2017	11/22/2017	Standard Class	HL-15040	Hunter Lopez	Consumer	United States	Newark	Delaware	19711	East
6827	CA-2016-118689	10/2/2016	10/9/2016	Standard Class	TC-20980	Tamara Chand	Corporate	United States	Lafayette	Indiana	47905	Central
8154	CA-2017-140151	3/23/2017	3/25/2017	First Class	RB-19360	Raymond Buch	Consumer	United States	Seattle	Washington	98115	West

Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
East	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS ...	11199.968	4	0.2	3919.9888
South	TEC-MA-10002412	Technology	Machines	Cisco TelePresenc...	22638.48	6	0.5	-1811.0784
East	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS ...	10499.97	3	0	5039.9856
Central	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS ...	17499.95	5	0	8399.976
West	TEC-CO-10004722	Technology	Copiers	Canon imageCLASS ...	13999.96	4	0	6719.9808

Practical 24

Code:

```
build.sbt  prac.scala x  superstore.csv

1  import org.apache.spark.sql.SparkSession
2  import org.apache.spark.sql.functions._
3
4  object prac extends App {
5      val spark = SparkSession.builder()
6          .appName("AverageSales")
7          .master("local[*]")
8          .getOrCreate()
9
10     val df = spark.read
11         .option("header", "true")
12         .option("inferSchema", "true")
13         .csv("src/main/resources/superstore.csv")
14
15     val result = df.groupBy("Category")
16         .agg(avg("Sales").alias("Average Sales"))
17
18     println("Average Sales by Category:")
19     result.show()
20
21     spark.stop()
22 }
23
```

Output:

```
+-----+-----+
|      Category|      Average Sales|
+-----+-----+
|Office Supplies|121.69225531914947|
|      Furniture| 353.4459311957568|
|      Technology| 454.5405475802047|
+-----+-----+
```